Technical Challenge: Project Aether
Distributed High-Load Task Orchestrator

Specialization: Rust Backend & Distributed Systems

April 2026

1 Легенда кейса

2026 год. Финтех-платформа *Aethelgard* столкнулась с критической деградацией производительности своей системы обработки отложенных платежей. Текущее решение на базе Python/Celery не обеспечивает требуемую пропускную способность в $500,000$ RPS и допускает потерю задач при кратковременных сбоях базы данных.

Вашей команде поручено разработать **Aether** — высокопроизводительный, отказоустойчивый оркестратор задач на языке **Rust**, использующий шардированную архитектуру хранения и гарантированную обработку через механизм Dead Letter Queue (DLQ).

2 Архитектура системы

2.1 Data Layer: Горизонтальное масштабирование

Система должна использовать **Application-level Sharding** для распределения нагрузки между несколькими независимыми инстансами PostgreSQL.

- **Consistent Hashing:** Распределение задач по шардам на основе идентификатора пользователя.
- **Replication:** Каждый шард должен иметь конфигурацию Master-Replica для обеспечения высокой доступности (HA).

2.2 Orchestration Layer (Rust)

Ядро системы должно быть реализовано с использованием асинхронного рантайма `tokio`.

- **Worker Nodes:** Реализация паттерна *Pull-based worker*, использующего `SELECT ... FOR UPDATE SKIP LOCKED` для обеспечения атомарности захвата задачи.
- **State Machine:** Каждая задача должна проходить жизненный цикл: `New` $\rightarrow$ `Processing` $\rightarrow$ (`Success` | `Failed` | `DLQ`).

3 Математическая модель шардирования

Для минимизации перераспределения данных при добавлении новых шардов используется кольцевое хеширование. Пусть $H(k)$ — хеш-функция. Множество шардов $S = \{s_1, s_2, \ldots, s_n\}$ отображается на кольцо идентификаторов $[0, 2^{32} - 1]$.

Задача T с ключом k назначается шарду:

$$TargetShard(T) = \text{argmin}_{s \in S}\{H(s) \mid H(s) \geq H(k)\} \tag{1}$$

Если $H(k) > \max(H(s))$, выбирается шард с минимальным хешем (замыкание кольца).

4 Интеграция с Kafka (Dead Letter Queue)

При невозможности выполнить задачу после N попыток (*Exponential Backoff Policy*), система обязана:

1. Атомарно удалить задачу из операционной БД (Postgres).
2. Сериализовать контекст задачи и ошибку в Protobuf/JSON.
3. Отправить сообщение в топик `aether.tasks.dlq` в **Apache Kafka** для последующего анализа и ручного перезапуска.

5 Технические требования

- ✓ **Язык:** Rust (edition 2021).
- ✓ **Frameworks:** Axum (API), Tonic (gRPC), SQLx (Database).
- ✓ **Infrastructure:** Docker Compose с описанием 3 шардов Postgres и кластера Kafka.
- ✓ **Observability:** Экспорт метрик в формате Prometheus (количество задач в очереди, latency выполнения).

6 Требования к Dashboard (Frontend)

Необходимо реализовать визуальный интерфейс для мониторинга (рекомендуется стиль *Glassmorphism*):

- Тепловая карта нагрузки на шарды (RPS per Shard).
- Интерфейс просмотра сообщений в Kafka DLQ.
- Статистика ошибок в реальном времени.